

Lecture 12

Deep Learning (Agent Perspective)

COMP532 – Machine Learning and Bioinspired Optimisation

Meng Fang, University of Liverpool



Why This Lecture?

Deep learning succeeds because we can *optimize* large, nonconvex models reliably.

Today:

- ▶ Credit assignment (backprop as the workhorse)
- ▶ Optimization dynamics (SGD and variants)
- ▶ Stability, scaling, and practical training heuristics

Agent perspective:

Learn representations $\Rightarrow$ learn policies / values / world models.

Roadmap

1. Learning objective and gradients
2. Backprop as credit assignment
3. SGD dynamics and optimization tricks
4. Stability issues (vanishing/exploding, curvature)
5. Regularization and generalization
6. Scaling and modern training patterns

Learning as Empirical Risk Minimization

We learn parameters θ by minimizing a loss over a dataset:

$$\min_{\theta} \mathcal{L}(\theta) \quad \text{where} \quad \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i).$$

Examples:

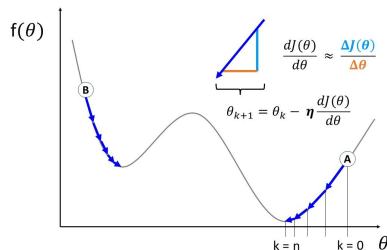
- ▶ Regression: $\ell(\hat{y}, y) = \frac{1}{2} \|\hat{y} - y\|^2$, where y is ground-truth label and $\hat{y} = f_{\theta}(x)$ is model output
- ▶ Classification: $\ell(\hat{p}, y) = -\log \hat{p}_y$ (cross-entropy), where $\hat{p}_y = P_{\theta}(Y = y \mid x)$ is the model's predicted probability of the correct class.

Gradients Drive Learning

Gradient descent updates parameters against the gradient:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t),$$

where η is the learning rate and $\nabla_{\theta} \mathcal{L}$ is the gradient of $\mathcal{L}$ with respect to θ .



Key question:

- How do we compute $\nabla_{\theta} \mathcal{L}$ efficiently for deep networks?

Answer:

Backpropagation = systematic application of the chain rule.

A Simple Computation Graph View

Consider a network as a composition of functions:

$$h_0 = x, \quad h_k = \phi_k(h_{k-1}; \theta_k), \quad \hat{y} = h_L, \quad \ell = \ell(\hat{y}, y).$$

We want:

$$\nabla_{\theta_k} \ell \quad \text{for all layers } k.$$

Backprop works by propagating *sensitivities* from output to input.

Single Neuron: Setup

Neuron:

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = \sigma(z).$$

Loss (example regression):

$$\ell = \frac{1}{2}(a - y)^2.$$

We need gradients (The derivative of the loss with respect to w and b):

$$\frac{\partial \ell}{\partial \mathbf{w}}, \quad \frac{\partial \ell}{\partial b}.$$

Single Neuron: Chain Rule

Compute step-by-step:

$$\frac{\partial \ell}{\partial a} = (a - y), \quad \frac{\partial a}{\partial z} = \sigma'(z), \quad \frac{\partial z}{\partial \mathbf{w}} = \mathbf{x}, \quad \frac{\partial z}{\partial b} = 1.$$

So:

$$\begin{aligned} \frac{\partial \ell}{\partial \mathbf{w}} &= \frac{\partial \ell}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial \mathbf{w}} = (a - y) \sigma'(z) \mathbf{x}, \\ \frac{\partial \ell}{\partial b} &= (a - y) \sigma'(z). \end{aligned}$$

Define the local error signal:

$$\delta = \frac{\partial \ell}{\partial z} = (a - y) \sigma'(z).$$

Then:

$$\nabla_{\mathbf{w}} \ell = \delta \mathbf{x}, \quad \frac{\partial \ell}{\partial b} = \delta.$$

Multi-Layer Networks: Backprop Recurrence

For layer k :

$$z_k = W_k h_{k-1} + b_k, \quad h_k = \sigma(z_k).$$

Let $\delta_k := \frac{\partial \ell}{\partial z_k}$. Then:

$$\delta_k = (W_{k+1}^\top \delta_{k+1}) \odot \sigma'(z_k),$$

(where $\odot$ is elementwise product).

Gradients:

$$\frac{\partial \ell}{\partial W_k} = \delta_k h_{k-1}^\top, \quad \frac{\partial \ell}{\partial b_k} = \delta_k.$$

Credit Assignment Interpretation

Backprop solves: *which internal units are responsible for the error?*

- ▶ Output layer: error is explicit from the loss.
- ▶ Hidden layers: error is *inferred* by propagating influence backward.

In agent terms:

- ▶ Representations are learned to support downstream objectives
- ▶ “Credit” flows from objective to features/policy/value parameters

Why Stochastic Gradient Descent (SGD)?

Full-batch gradient:

$$\nabla \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\theta)$$

is expensive for large N .

Mini-batch SGD:

$$\mathbf{g}_t = \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \nabla \ell_i(\theta_t), \quad \theta_{t+1} = \theta_t - \eta \mathbf{g}_t.$$

SGD introduces noise that can help escape sharp minima / saddle regions.

Momentum

Momentum accumulates a velocity:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \mathbf{g}_t, \quad \theta_{t+1} = \theta_t - \eta \mathbf{v}_{t+1}.$$

Intuition:

- ▶ Averages gradients to reduce variance
- ▶ Accelerates along consistent directions
- ▶ Dampens oscillations in high curvature directions

Adaptive Methods - Adam (Note: This will not be tested in the exam.)

Adam maintains first and second moment estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Bias-corrected:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Rule of thumb:

- ▶ Adam is robust for many problems
- ▶ SGD(+momentum) can generalize better in some regimes

Learning Rate is the Main Knob

Learning rate η largely determines:

- ▶ Stability (diverge vs converge)
- ▶ Speed of training
- ▶ Final performance (often via implicit regularization)

Common schedules:

- ▶ Step decay
- ▶ Cosine decay
- ▶ Warmup + decay (common for Transformers/LLMs)

Practical note:

If training is unstable, try reducing η first.

Vanishing / Exploding Gradients

In deep compositions, gradients involve products of Jacobians: (The gradient with respect to x is proportional to the product of Jacobians across layers.)

$$\nabla_x f(x) \propto \prod_{k=1}^L J_k.$$

If $\|J_k\| < 1$ on average:

gradients vanish

If $\|J_k\| > 1$ on average:

gradients explode.

Mitigations:

- ▶ Better activations (ReLU/GELU)
- ▶ Initialization (He/Xavier)
- ▶ Normalization (BatchNorm / LayerNorm)
- ▶ Residual connections

Initialization Matters (Note: This will not be tested in the exam.)

Goal: keep signal scales stable across layers.

For ReLU networks (He init):

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

For tanh/sigmoid (Xavier):

$$W_{ij} \sim \mathcal{N}\left(0, \frac{1}{n_{\text{in}}}\right)$$

Bad initialization can cause:

- ▶ Early saturation (tiny gradients)
- ▶ Numerical blow-up
- ▶ Slow convergence

Normalization Stabilizes Training (Note: This will not be tested in the exam.)

BatchNorm (for CNN/MLP) normalizes activations in a mini-batch.

LayerNorm (standard for Transformers) normalizes per token / per feature.

Effect:

- ▶ Controls activation scales
- ▶ Makes optimization less sensitive to η
- ▶ Enables deeper networks

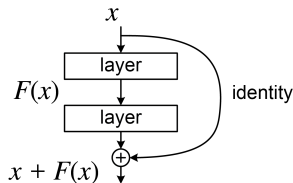
Takeaway:

Normalization is an optimization tool, not just “regularization”.

Residual Connections

Residual block:

$$h_{k+1} = h_k + F(h_k).$$



Why it helps:

- ▶ Provides identity paths for gradient flow
- ▶ Reduces effective depth for optimization
- ▶ Enables very deep networks (e.g., ResNets, Transformers)

Overfitting and Generalization

Even when training loss is low, test loss may be high.

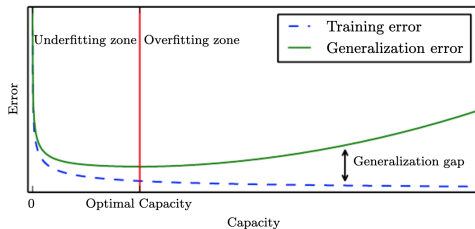
We care about:

$$\mathbb{E}_{(x,y) \sim \mathcal{D}} \ell(f_{\theta}(x), y)$$

not just training error.

Generalization depends on:

- ▶ Data quality and diversity
- ▶ Model capacity
- ▶ Optimization dynamics (implicit bias)



Explicit Regularization

Common tools:

- ▶ Weight decay (L2):

$$\min_{\theta} \mathcal{L}(\theta) + \lambda \|\theta\|_2^2$$

- ▶ Dropout: randomly zero activations during training
- ▶ Data augmentation: enlarge effective dataset
- ▶ Early stopping

Note: in modern large models, *data* is often the best regularizer.

Implicit Regularization of SGD

Empirical observation:

- ▶ SGD tends to find “simple” solutions among many interpolating solutions
- ▶ Learning rate and batch size influence this bias

Heuristic:

Larger batch $\Rightarrow$ less gradient noise $\Rightarrow$ more careful tuning needed.

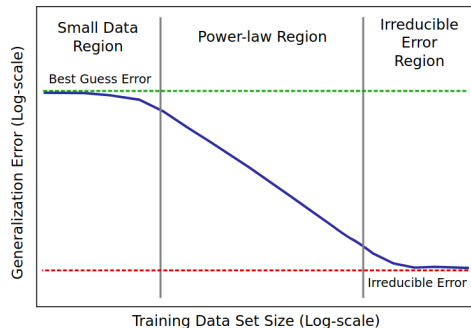
Training - Code

```
# -----  
# Train / Eval utils  
# -----  
@torch.no_grad()  
def evaluate(model, loader, device):  
    model.eval()  
    correct, total, loss_sum = 0, 0, 0.0  
    criterion = nn.CrossEntropyLoss()  
    for x, y in loader:  
        x, y = x.to(device), y.to(device)  
        logits = model(x)  
        loss = criterion(logits, y)  
        loss_sum += loss.item() * x.size(0)  
        pred = logits.argmax(dim=1)  
        correct += (pred == y).sum().item()  
        total += y.numel()  
    return loss_sum / total, correct / total  
  
def train(model, train_loader, test_loader, device, epochs=5, lr=1e-3):  
    model.to(device)  
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)  
    criterion = nn.CrossEntropyLoss()  
  
    for epoch in range(1, epochs + 1):  
        model.train()  
        for x, y in train_loader:  
            x, y = x.to(device), y.to(device)  
            optimizer.zero_grad(set_to_none=True)  
            logits = model(x)  
            loss = criterion(logits, y)  
            loss.backward()  
            optimizer.step()  
  
    test_loss, test_acc = evaluate(model, test_loader, device)
```

Scaling Laws (High-Level)

For large models, performance often follows smooth trends with:

- ▶ Model size
- ▶ Data size
- ▶ Compute budget



Practical implication:

- ▶ Optimization must be stable at scale
- ▶ Schedules (warmup/decay), normalization, and residuals become essential

Big Picture

Learning =

- ▶ Define loss
- ▶ Compute gradients
- ▶ Update parameters
- ▶ Repeat

Neural Networks in Agents

In agents:

- ▶ Loss may come from labels
- ▶ Or from rewards
- ▶ Or from next-token prediction

But optimization principle is the same.

Key Takeaways

- ▶ Learning means minimizing a loss.
- ▶ Gradients tell us how to adjust parameters.
- ▶ Backprop spreads error backward.
- ▶ Training is iterative improvement.

Next Lecture

Next:

- ▶ How architecture shapes learning
- ▶ Convolution and structure